

# STAT 663 Project 2 - Pills Inside Bottles

Wenxin Guo, Justin Kao

Statistical Science  
Duke University

2026/09/25



- 1 Motivation
- 2 Dataset Introduction
- 3 Selection of Embedding Technique
- 4 Comparative Analysis of Classification Models
- 5 Conclusion

# Main Goal of the Study

## Performance Evaluation of a Prescription Medication Image Classification Model

Our primary objective is to assess the efficacy and accuracy of a machine learning model designed for the classification of prescription medication images. This main idea of our ML classification focuses on evaluating how well the model can identify and categorize images of pills within medication bottles based on a predefined dataset. Ultimately, we can facilitate the automatic identification of pills to increase efficiency and reduce dispensing error of pills in pharmacy.

The evaluation metrics will include accuracy, precision, recall, and F1 score, aiming to provide a comprehensive understanding of the model's performance in a real-world healthcare setting. By analyzing these outcomes, we aim to contribute valuable insights into the development and refinement of AI-driven tools in medical image analysis, particularly in the area of medication recognition and classification.

- 1 Motivation
- 2 Dataset Introduction**
- 3 Selection of Embedding Technique
- 4 Comparative Analysis of Classification Models
- 5 Conclusion

# Dataset Overview : Pills Inside Bottles

- **'image'** : An image of pills inside a medication bottle.
- **'id'** : A unique identifier for each image.
- **'ndc'** : The National Drug Code, which serves as a unique identifier for medications, implying the specific type of medication pictured.

---

```
dataset = load_dataset("gwenxin/pills_inside_bottles", trust_remote_code=True)
```

```
dataset
DatasetDict({
  train: Dataset({
    features: ['image', 'id', 'ndc'],
    num_rows: 8393
  }),
  test: Dataset({
    features: ['image', 'id', 'ndc'],
    num_rows: 2786
  }),
  validation: Dataset({
    features: ['image', 'id', 'ndc'],
    num_rows: 2776
  })
})
```

# Dataframe : A Specific Example - No. 663

```
dataset['train'][663]['id'] # image id  
'97189'  
dataset['train'][663]['ndc'] # ndc code  
'53489-0156'  
import matplotlib.pyplot as plt  
image_example = dataset['train'][663]['image']  
plt.imshow(image_example) # image
```



# Dataframe : Navigating Examples

```

from PIL import Image
from IPython.display import display

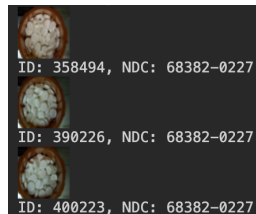
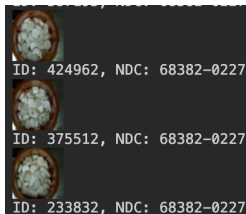
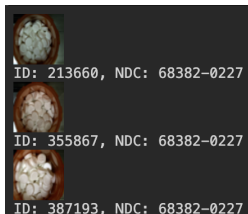
# Set the desired size
desired_size = (50, 50) # width, height in pixels

for index, row in train_df.head(1000).iterrows():
    # Resize the image
    display_image = row['image'].resize(desired_size)

    # Display the resized image
    display(display_image)

    # Print the other data associated with the image
    print(f"ID: {row['id']}, NDC: {row['ndc']}")

```



- 1 Motivation
- 2 Dataset Introduction
- 3 Selection of Embedding Technique**
- 4 Comparative Analysis of Classification Models
- 5 Conclusion

# Potential Embedding Methods



In Keras Applications, from the 38 models available on the Keras website, we selected five that performed best based on the classifiers we intended to use. We tested 24 models; the others caused our Colab environment to crash!!!

| Model             | Size (MB) | Top-1 Accuracy | Top-5 Accuracy | Parameters | Depth |
|-------------------|-----------|----------------|----------------|------------|-------|
| ResNet50          | 98        | 74.9%          | 92.1%          | 25.6M      | 107   |
| DenseNet121       | 33        | 75.0%          | 92.3%          | 8.1M       | 242   |
| InceptionResNetV2 | 215       | 80.3%          | 95.3%          | 55.9M      | 449   |
| EfficientNetV2L   | 479       | 85.7%          | 97.5%          | 119.0M     | -     |
| EfficientNetV2M   | 220       | 85.3%          | 97.4%          | 54.4M      | -     |

Table – Performance metrics of different models.

# Optimal Embedding Techniques for Pill Classification

Upon evaluating various embedding methodologies for our pill classification model, we have ascertained that the combination of **EfficientNetV2M with an SVM classifier** yields the highest accuracy, achieving an accuracy of **0.97**.

> ResNet50 / Accuracy: (RandomForest: 0.8560660445082555)(Logistic: 0.914572864321608)(SVM: 0.9149318018664753)

▶ [Show code](#)

> DenseNet121/ Accuracy: (RandomForest: 0.8765254845656856)(Logistic: 0.905240488155061)(SVM: 0.9012921751615219)

▶ [Show code](#)

> Inception-ResNet v2 / Accuracy: (RandomForest: 0.8305814788226848)(Logistic: 0.9178033022254127)(SVM: 0.9228284278535535)

▶ [Show code](#)

> EfficientNetV2L / Accuracy: (RandomForest: 0.8241206030150754)(Logistic: 0.9142139267767408)(SVM: 0.9328786791098349)

▶ [Show code](#)

> EfficientNetV2M / Accuracy: (RandomForest: 0.9091888011486001)(Logistic: 0.9601579325197416)(SVM: 0.9712849964106246)

▶ [Show code](#)

# EfficientNetV2M - Initialization & Configuration

---

```

# @title EfficientNetV2M /
Accuracy:
(RandomForest: 0.9091888011486001)
(Logistic: 0.9601579325197416)
(SVM: 0.9712849964106246)
from tensorflow.keras.applications.efficientnet_v2 import EfficientNetV2M, preprocess_input
import tensorflow as tf
import numpy as np

# Initialize the EfficientNetV2M model
model = EfficientNetV2M(
    include_top=False, # Exclude the top softmax layer
    weights='imagenet', # Use weights pre-trained on ImageNet
    pooling='avg' # Include global average pooling to reduce the output to a 1-D vector
)

```

---

# EfficientNetV2M - Processing

---

```
def processing(dataset, data_type, model, batch_size):
    num_samples = len(dataset[data_type])
    X = []
    for i in range(0, num_samples, batch_size):
        batch_X = dataset[data_type][i:i+batch_size]['image']
        processed_X = []
        for x in batch_X:
            # Convert the image to a numpy array and resize it
            # EfficientNetV2M expects images of size 480x480
            x_array = np.array(x)
            x_resized = tf.image.resize(x_array, (480, 480))
            # Preprocess the image using the appropriate preprocessing function for EfficientNet
            processed_image = preprocess_input(x_resized)
            processed_X.append(processed_image)

        # Use the model to predict the features
        embedding = model.predict(np.array(processed_X))
        X.extend(embedding)

    # Convert the list of features to a numpy array
    new_X = np.array(X)
    return new_X
```

---

- 1 Motivation
- 2 Dataset Introduction
- 3 Selection of Embedding Technique
- 4 Comparative Analysis of Classification Models**
- 5 Conclusion

# Supervised Algorithms

We plan to utilize the embedded data for classification and aim to predict the **National Drug Code("npc")** based on the provided data and the chosen model. The models under consideration include Random Forest, Support Vector Machine (SVM), Decision Tree, Naive Bayes, Multi-Layer Perceptron (MLP), Extra Tree, and Stochastic Gradient Descent (SGD) Classifier.

| Embedding Method       | Random Forest | Logistic      | SVM           |
|------------------------|---------------|---------------|---------------|
| ResNet50               | 0.8566        | 0.9145        | 0.9149        |
| DenseNet121            | 0.8765        | 0.9052        | 0.9013        |
| Inception-ResNet v2    | 0.8306        | 0.9178        | 0.9228        |
| EfficientNetV2L        | 0.8241        | 0.9142        | 0.9329        |
| <b>EfficientNetV2M</b> | <b>0.9092</b> | <b>0.9602</b> | <b>0.9713</b> |

Table – Accuracy of classifiers using different embedding methods.

# Basic Algo. 1 - Random Forest with Tuning Parameters

---

```
initial_val_accuracy = accuracy_score(y_validation, y_val_pred)
print(f"Initial validation accuracy: {initial_val_accuracy}")

best_n_estimators = 100
best_val_accuracy = initial_val_accuracy

for n_estimators in [50, 100, 200, 300]:
    model = RandomForestClassifier(n_estimators=n_estimators, random_state=42)
    model.fit(X_train, y_train)
    y_val_pred = model.predict(X_validation)
    val_accuracy = accuracy_score(y_validation, y_val_pred)

    if val_accuracy > best_val_accuracy:
        best_val_accuracy = val_accuracy
        best_n_estimators = n_estimators
```

---

Initial validation accuracy : 0.9074207492795389

Tuned model test accuracy : 0.91816223977028

# Basic Algo. 2 - Logistic Regression Classifier

Accuracy : 0.95908111988514

Classification Report :

| Class               | Precision | Recall | F1-Score | Support |
|---------------------|-----------|--------|----------|---------|
| 00378-0208          | 0.95      | 0.97   | 0.96     | 200     |
| 00378-3855          | 0.98      | 1.00   | 0.99     | 42      |
| 00591-0461          | 0.92      | 0.94   | 0.93     | 48      |
| ⋮                   | ⋮         | ⋮      | ⋮        |         |
| 69097-0128          | 0.90      | 0.89   | 0.89     | 200     |
| 69315-0904          | 0.97      | 0.98   | 0.98     | 118     |
| 69315-0905          | 0.96      | 0.99   | 0.97     | 108     |
| <b>Accuracy</b>     |           |        | 0.96     | 2786    |
| <b>Macro Avg</b>    | 0.96      | 0.95   | 0.95     | 2786    |
| <b>Weighted Avg</b> | 0.96      | 0.96   | 0.96     | 2786    |

# Optimal Algo - Support Vector Machine (SVM) Classifier :

SVM Accuracy : 0.9712849964106246

SVM Classification Report :

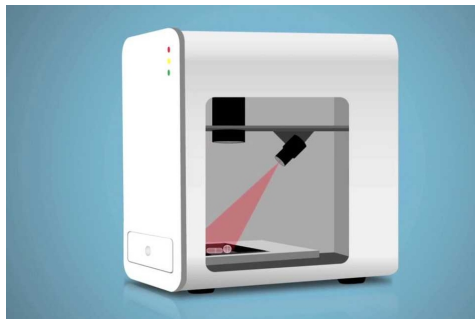
| Class               | Precision | Recall | F1-Score | Support |
|---------------------|-----------|--------|----------|---------|
| 00378-0208          | 0.96      | 0.98   | 0.97     | 200     |
| 00378-3855          | 0.98      | 1.00   | 0.99     | 42      |
| 00591-0461          | 0.92      | 0.98   | 0.95     | 48      |
| ⋮                   | ⋮         | ⋮      | ⋮        | ⋮       |
| 69097-0128          | 0.93      | 0.89   | 0.91     | 200     |
| 69315-0904          | 0.99      | 0.97   | 0.98     | 118     |
| 69315-0905          | 0.99      | 1.00   | 0.97     | 108     |
| <b>Accuracy</b>     |           |        | 0.97     | 2786    |
| <b>Macro Avg</b>    | 0.97      | 0.97   | 0.97     | 2786    |
| <b>Weighted Avg</b> | 0.97      | 0.97   | 0.97     | 2786    |

- 1 Motivation
- 2 Dataset Introduction
- 3 Selection of Embedding Technique
- 4 Comparative Analysis of Classification Models
- 5 Conclusion**

# Conclusion

In the realm of pill image classification, our goal is to assist pharmacists in reducing the error rates associated with dispensing medication. By incorporating an additional verification step where the scanner reviews the contents of the vials, we anticipate a significant reduction in incidents of incorrect dosages and medication errors.

Although our model, leveraging SVM with EfficientNetV2M, achieves an impressive accuracy of 97.13%, **in medical applications, even a 99% accuracy rate** may not be sufficient due to the high stakes involved. Therefore, it is crucial to continue increasing the size of training set, refining our approach to image-based models and classifier combinations to enhance reliability and safety in pharmaceutical practices.



MIT News : "An end to drug errors? Mint Solutions tackles medication errors with scanning system that ensures patients get the right pills."

# The End

When you forget how much medication you were administered and try to take an appropriate amount of it



## Thank You for Your Attention !

Although we work on pill image classification, we hope everyone remains healthy and doesn't need to take medication.

Source : Reddit r/meme